Coventry University

7009SCN-Cloud Infrastructure and Operations

Coursework

SEGUN AKINSOYINU - 16498040
12-15-2025

A.1 Container & Image Security and Build Process

A.1.1 Evaluation of the Current Manual Build Process — Risks and Limitations

The original prototype relied on a Makefile-driven workflow for dbserver/ and webserver/, with developers manually invoking targets to build and run each container. The environment used CentOS 7–based images and ad hoc Docker commands for networking and lifecycle management, with no central orchestration.

Current manual process (prototype)

- **Database container** – CentOS 7–based MariaDB, manually initialised with SQL scripts.
- **Web container** – CentOS 7 with Nginx, PHP-FPM and an SSH server.
- **Networking** – Manually created bridge networks and IP allocation.
- **Lifecycle** – No automated dependency handling or health checks; containers started/stopped in arbitrary order.

This approach led to inconsistent builds, configuration drift and no enforcement of startup order or policy (Docker Inc., 2024).

Security issues identified

1. **Outdated base images**
   CentOS 7 reached end of life in June 2024, meaning no further upstream security patches. Continuing to use it significantly increases exposure to unpatched vulnerabilities.
2. **Privileged containers and root execution**
   The database container was started with --privileged=true, and both containers ran as root. This breaks container isolation, grants broad kernel access and violates the principle of least privilege (NIST, 2020).
3. **Excessive service and port exposure**
   The web container exposed SSH (22) and the Docker daemon port (2375) in addition to the HTTP port. Running SSH in application containers is strongly discouraged, as it introduces a full remote-execution channel and bypasses normal access controls (CIS Docker Benchmark, 2023).
4. **Weak authentication and secrets handling**
   Hard-coded credentials (e.g. MYSQL_ROOT_PASSWORD) and default passwords were embedded in Makefiles and environment variables, making them easy to leak via version control or logs (OWASP, 2021).

5. **No resource constraints**
   Containers were started without CPU or memory limits, allowing a single compromised or buggy service to exhaust host resources and cause denial-of-service.

6. **Lack of declarative configuration (no IaC)**
   No single, version-controlled specification described services, ports, networks, volumes and dependencies. Auditing, rollback and consistent environment reproduction were therefore difficult (HashiCorp, 2024).

Taken together, these weaknesses expanded the attack surface significantly. Research highlights outdated base images, privileged execution and excessive exposure as leading causes of container compromise (Sharma & Sood, 2021; Alzahrani et al., 2022).

A.1.2 Migration to Declarative Builds (Docker Compose)

To address operational and security issues in the manual process, the system is migrated to a declarative Docker Compose–based workflow. A hardened Compose file (e.g. docker-compose.hardened.yml) defines services, networks, volumes, health checks and security options as Infrastructure as Code.

Security benefits

- **Declarative security configuration**
  Security controls such as non-root users, capability drops, read-only filesystems, secrets, resource limits and health checks are codified in the Compose file and subject to version control and code review (Docker Compose, 2024).

- **Consistent environment reproduction**
  The same Compose configuration can be used for development and staging, reducing configuration drift and “it works on my machine” issues (NIST, 2020).

- **Automated dependency management**
  Service dependencies and depends_on/health checks ensure that the database is healthy before the web application starts, reducing startup race conditions.

- **Network segmentation**
  User-defined networks separate front-end and back-end traffic and strictly limit inter-container communication, reducing lateral movement opportunities.

Operational benefits

- **Single-command deployment and teardown**
  docker compose up -d and docker compose down provide deterministic lifecycle management.
- **Version-controlled infrastructure**
  Changes to service definitions, ports and policies are tracked alongside application code, enabling rapid rollback.
- **Standardised lifecycle management**
  Operators use a single set of commands and scripts (manage-secure.sh) to build, scan, deploy, inspect logs and tear down the environment, reducing human error.

These changes align with research showing that declarative orchestrators reduce configuration drift and support more predictable, automated operations (Goel et al., 2022).

A.1.3 Container Image Hardening Practices

Minimal base images and multi-stage builds

The original CentOS 7 images (hundreds of MB with generic tools) are replaced by minimal, actively maintained images:

- **Web** – Alpine-based Nginx/PHP-FPM images and multi-stage builds: a build stage with dependencies and tooling, and a stripped runtime stage containing only Nginx, PHP-FPM and the built artefacts.
- **Database** – Official MariaDB images based on Alpine or slim distributions, with regular security updates.

Multi-Stage Build Implementation
Multi-stage builds ensure that compilers, package managers and other build-time tools never appear in the final runtime image. This reduces image size, scan time and attack surface, and prevents accidental leakage of build-time secrets. In practice, this reduced the web image from roughly ~600 MB to ~50 MB and removed numerous unused binaries.

Non-root users and permissions

Dedicated service accounts are created for each container, for example:

```
RUN addgroup -g 1001 -S appgroup \
 && adduser -u 1001 -S appuser -G appgroup
```

USER 1001:1001

- Nginx and PHP-FPM processes run as unprivileged users (e.g. nginx and wwwapp) with restricted home directories.
- MariaDB runs as the mysql user.
- File permissions and ownership are adjusted so only these service accounts can write to application and data directories.

This enforces least privilege and mitigates the impact of a compromise (Sharma & Sood, 2021).

Read-only root filesystem and controlled write paths

Where the application allows it, the container root filesystem is mounted read-only. Writable paths (Nginx temp directories, logs, upload directories, database data) are mapped to:

- tmpfs volumes for ephemeral temporary files, and
- named volumes for persistent data.

During hardening, the docker-entrypoint-hardened.sh script was updated to create required Nginx temporary directories at startup, ensuring that read-only root remains compatible with Nginx/PHP-FPM operation.

Resource constraints

CPU and memory limits are applied to containers in the Compose configuration, ensuring that misbehaving or malicious workloads cannot exhaust host resources. This is particularly important in shared environments and forms part of defence against denial-of-service scenarios.

Resolved functional issues

While hardening, several operational problems were addressed:

- **Nginx temporary directories** – Created at startup to avoid runtime failures under read-only root.
- **PHP-FPM connectivity** – Configuration corrected to ensure consistent Nginx → PHP-FPM communication.
- **Database connectivity** – Confirmed and corrected inter-container networking so the web tier can reliably reach the database service on the internal network.

A summary of before/after state is shown below.

Security Aspect	Original State	Hardened State	Effect
Base Images	CentOS 7 (EOL)	Alpine-based, supported images	Reduced inherited CVEs
Build Pattern	Single-stage, full toolchain	Multi-stage builds	Smaller, cleaner runtime images
Privileges	Root + --privileged	Dedicated non-root users, no privileged flag	Least-privilege enforced
Filesystem	Fully writable	Read-only root with tmpfs/volume exceptions	Integrity protection
Resource Limits	None	CPU/memory limits per container	DoS risk reduction
Secrets	Hard-coded envs / Makefile values	Docker secrets and file-based injection	Lower credential leakage risk

These measures follow best practices for container image hardening (Tu et al., 2021; Singh et al., 2020).

A.1.4 Vulnerability Scanning Strategy

A structured vulnerability management pipeline is implemented around **Trivy** as the primary scanner, with **Snyk** available as a complementary tool for deeper application and licence analysis.

Trivy integration

Trivy is used to scan:

- **OS packages** – Vulnerabilities in base OS layers.
- **Language dependencies** – Vulnerable libraries in the application stack.
- **Configuration** – Insecure Dockerfiles and Compose files.
- **Secrets** – Hard-coded credentials in source and configuration.

 # Image scanning (local or CI)

```
trivy image --severity HIGH,CRITICAL your-image:tag
```

 # Configuration scanning

```
trivy config .
```

 # CI gate: fail on high/critical

```
trivy image --exit-code 1 --severity HIGH,CRITICAL your-image:tag
```

This supports both local developer checks and automated quality gates in CI/CD.

Snyk (complementary, where used)

Snyk can be used to provide additional application-level insight, licence compliance and ongoing monitoring.

```
snyk container test your-image:tag
snyk container monitor your-image:tag
```

Where integrated, this provides continuous monitoring for newly disclosed vulnerabilities.

Scanning pipeline

The overall strategy follows a multi-stage approach:

1. **Pre-commit / local** – Optional local scans by developers before pushing.
2. **Build-time** – Automated Trivy scans in CI after image build; pipelines fail on HIGH/CRITICAL issues.
3. **Configuration scans** – Trivy config scans on Dockerfiles/Compose to detect bad practices (root, unpinned tags, exposed sensitive ports).
4. **Periodic registry scans** – Registry-based re-scanning of stored images when CVE databases update.
5. **Runtime / compliance (recommended)** – Periodic scans of running images and CIS-benchmark-style checks on the environment.

6. **Remediation Workflow**

Severity	Required Action
Critical/High	Block deployment and patch or update base image immediately
Medium	Fix within SLA
Low	Track for routine maintenance

This pipeline directly addresses empirical findings that container images often ship with large numbers of inherited vulnerabilities and misconfigurations (Alzahrani et al., 2022; Aqua Security, 2024; NIST, 2020).

A.1.5 Principle of Least Privilege for Application Processes and Runtime

Application users and capabilities

As implemented in the hardened design.

- All application containers run as non-root users, defined via USER directives and created with minimal privileges.
- Linux capabilities are dropped by default, with only NET_BIND_SERVICE added where required for Nginx to bind to port 80:

```
cap_drop:
 - ALL
cap_add:
 - NET_BIND_SERVICE
```

- no-new-privileges: true is applied at container runtime level where supported, preventing processes from elevating privileges via SUID binaries (Docker Security Best Practices, 2024).

Filesystem and runtime controls

- Root filesystems are configured read-only, with only necessary paths writable through dedicated volumes or tmpfs.

- Docker secrets are used for sensitive data such as database passwords, mounted as root-readable files with strict permissions rather than passed as long-lived environment variables.
- Resource constraints (CPU/memory) prevent unbounded resource consumption by a compromised process.

Network and HTTP-layer controls

- Only required ports are exposed externally (HTTP on port 80); SSH and Docker daemon access are removed from the web container.
- Inter-container communication is restricted to the minimum required paths (web → database).
- At the web layer, standard HTTP hardening measures (such as security headers and rate limiting) can be applied via Nginx configuration to mitigate common web-layer attacks and brute-force attempts.

Research shows that container escape techniques frequently rely on excessive capabilities, writable root filesystems, privileged execution and root processes (Sharma & Sood, 2021). The implemented measures significantly reduce these conditions and limit the blast radius of any compromise.

A.2 Docker Host and Daemon Security

A.2.1 Threats Targeting the Docker Host and Daemon

(Unchanged conceptually – still covers docker.sock abuse, exposed daemon, unpatched host and isolation weaknesses.)

A.2.2 Recommended Docker Daemon and Host Controls

In addition to the earlier controls (restricted socket access, patching, icc=false, userns-remap, live-restore), your hardening plan includes concrete daemon.json settings for:

- **TLS securing of remote access** – Enabling tls, tlsverify, and providing certificate/key paths to ensure that any TCP listener (e.g. tcp://0.0.0.0:2376) is authenticated and encrypted.
- **Default resource limits** – Setting default-ulimits and default-shm-size to enforce sensible file-descriptor limits and shared memory sizes cluster-wide.

Where root access to the host is available, these settings should be applied to enforce encrypted daemon communication and baseline resource controls.

User namespace remapping configuration (adding dockremap ranges to /etc/subuid and /etc/subgid and setting "userns-remap": "default") further ensures that root inside containers does not map to root on the host, adding another layer of isolation.

A.2.3 Leveraging Linux Kernel Security Mechanisms

Beyond enabling AppArmor/SELinux and seccomp in general, your hardening design includes **custom profiles** for sensitive services (e.g. a docker-nginx AppArmor profile), and references a custom seccomp policy JSON. These profiles:

- Remove dangerous capabilities and syscalls that Nginx/PHP-FPM do not require.
- Deny write access to sensitive kernel and proc/sys paths.

This moves from generic “enable MAC and seccomp” advice to concrete, application-specific confinement.

AppArmor Profile Example:

```
#include <tunables/global>

profile docker-nginx flags=(attach_disconnected,mediate_deleted) {
 #include <abstractions/base>


 capability setuid,
 capability setgid,
 capability net_bind_service,


 deny /proc/sys/** w,
 deny /sys/** w,
}
```

Enable user namespace remapping

```
echo 'dockremap:165536:65536' >> /etc/subuid
echo 'dockremap:165536:65536' >> /etc/subgid
```

Configure daemon

```
echo '{"userns-remap": "default"}' > /etc/docker/daemon.json
```

Security Context Configuration

```
security_opt:
 - no-new-privileges:true
 - apparmor:docker-nginx
 - seccomp:seccomp.json
```

A.3 Logging, Monitoring and Operational Usability

A.3.1 Local and layered logging

You already describe persistent logs via host-mounted directories. Building on that, your design explicitly considers **multi-layer logging**:

- **Application logs** – PHP error logs, Nginx access/error logs.
- **Container logs** – stdout/stderr via the Docker JSON log driver.
- **System logs** – Docker daemon and host OS logs.
- **Security logs** – Authentication attempts and privilege escalation events.

This layered view supports both operational debugging and security investigation.

A.3.2 Log inspection and centralisation path

Local inspection with docker compose logs and wrapper scripts (manage-secure.sh) standardises basic access. For production-scale monitoring, you provide a concrete ELK Stack compose example (Elasticsearch, Logstash, Kibana) as a reference implementation:

- Elasticsearch as the central log store.
- Logstash to ingest and parse logs from Docker hosts.
- Kibana to visualise and search logs.

This bridges from the current JSON-file logs to a fully centralised logging architecture without overcomplicating the local development environment.

A.3.3 Metrics, Alerts and Security-Focused Monitoring

Comprehensive observability requires both performance and security-oriented metrics. Prometheus provides metrics scraping across containers and hosts, while Grafana supports dashboarding and alerting.

Performance metrics include:

- CPU utilisation and throttling
- Memory usage and OOM events
- Disk I/O and storage pressure

- Network throughput and error rates

Application-level metrics include:

- Request latency and HTTP error rates
- PHP-FPM pool utilisation
- Database query performance

Security-relevant metrics include:

- Failed login counters
- Abnormal request volume spikes
- Unexpected inter-container communication patterns
- Seccomp/AppArmor violation events (if enforced)

.

A.3.4 Operational Usability and Automation Considerations

Transitioning from manual Makefile-driven processes to declarative Compose-based orchestration improves operational reliability:

- Single-command lifecycle control for deployment, teardown and updates
- Automated dependency orchestration via health checks and explicit depends on rules
- Standardised operator workflows enforced through helper scripts
- Reduced configuration drift as all service definitions are version-controlled

These usability enhancements streamline operations while embedding security controls directly into the daily workflow, minimising the chance of misconfiguration or human error.

A.3.5 Incident Response and Forensic Readiness

A hardened environment must support rapid detection, containment and analysis of security incidents.

Recommended incident-response capabilities include:

- Rapid anomaly detection using combined log, metric and alert data
- Container isolation or reconstruction from known-good images in suspected compromise scenarios
- Traffic blocking using updated network rules or container-level isolation
- Forensic snapshotting of container filesystems and relevant logs
- Structured log retention policies (e.g., 90-day minimum for security logs)
- Audit trails via version-controlled Compose and Docker artefacts

SIEM Integration (Recommended for Production Environments)

For organisations requiring enhanced threat detection and compliance, logs and security telemetry should be forwarded to a Security Information and Event Management (SIEM) platform such as Splunk, Elastic SIEM, or Wazuh.
A SIEM enables:

- Correlation of events across hosts, containers, networks and users
- Detection of multi-stage or lateral movement attacks
- Long-term, compliance-aligned log retention
- Centralised incident triage and case management
- Behavioural and anomaly-based detection beyond static thresholds

SIEM integration typically consumes logs via Logstash and may incorporate Prometheus security metrics as part of a unified threat-monitoring pipeline.

A.3.6 Security–Usability Trade-offs

Security hardening introduces certain operational constraints:

- Read-only roots require configuration updates through new image builds
- Non-root execution mandates disciplined file-permission management
- Secrets management shifts credentials from environment variables to secure files, requiring stricter access controls
- Enhanced observability tooling increases configuration complexity

However, these trade-offs are mitigated by predictable automation, declarative configuration, and structured operational procedures. The resulting system provides a

significantly improved security posture while remaining operationally feasible for administrators.

Part B – Secure Implementation Summary

Part B implements a fully hardened Docker-based deployment. Secure Dockerfiles are used for the database and web services to minimise attack surface and enforce non-root execution. A hardened Docker Compose configuration applies runtime security controls, including restricted privileges, dropped capabilities, read-only filesystems, secrets management, resource limits, and health checks. Service-level configuration files further harden MySQL, Nginx, and PHP by disabling unsafe features and reducing information exposure. The manage-secure.sh script automates secure build, scan, deployment, and operational tasks, ensuring consistent and error-free system management.

OneDrive practical implementation link

- **hdhjkdllddldkhdh**

C Production Deployment Strategy, Scalability and Cost

C.1 Platform Selection and Justification

The hardened system now uses Docker Compose for consistent, secure local deployment; however, Compose cannot provide the scalability, resilience, automated failover or cluster-level security controls required in production. Running containers on a single VM still limits availability, introduces configuration drift and prevents enforcement of centralised runtime policies. A managed cloud orchestration platform is therefore the required next step.

Three cloud deployment models were evaluated:

Infrastructure as a Service (IaaS) offers VM-level control but no native orchestration. Scaling, failover, health checks and rolling updates must be manually engineered, making it unsuitable for high-availability production workloads.

Platform as a Service (PaaS) option such as Elastic Beanstalk or Azure App Service simplify deployments but restrict low-level container security controls. Key hardening measures established in Part A non-root execution, dropped capabilities and read-only filesystems cannot be consistently applied, and PaaS limits fine-grained network policies required for segmentation.

Container as a Service (CaaS) with Kubernetes via AWS EKS, Azure AKS or Google GKE provides managed control planes, declarative deployments, auto-healing, horizontal and cluster autoscaling, and integrated security features. Kubernetes extends the Docker Compose model into a cloud-ready architecture, supporting the hardened runtime controls and operational patterns already implemented.

CaaS therefore represents the natural progression from Compose-based development to a scalable, resilient and security-aligned production environment.

Justification for AWS EKS

AWS Elastic Kubernetes Service (EKS) is selected based on the following attributes:

- **Managed Kubernetes control plane**, reducing operational overhead and improving fault tolerance.
- **Deep integration with AWS IAM**, enabling IAM Roles for Service Accounts (IRSA) to enforce least privilege for pods an essential extension of the Part A security model.
- **Integration with VPC, CloudWatch, ECR and KMS**, enabling encrypted secrets, structured logs and secure image storage.
- **Support for PodSecurityContext enforcement**, enabling non-root execution, read-only filesystems and capability dropping in Kubernetes manifests.
- **Native autoscaling mechanisms** (Horizontal Pod Autoscaler, Cluster Autoscaler).
- **Operational maturity**, ecosystem tooling and AWS's strong multi-AZ infrastructure.

Alternative cloud orchestrators AKS and GKE offer rich Kubernetes implementations, but would introduce vendor fragmentation and additional tooling overhead. ECS Fargate provides strong operational simplicity but lacks ecosystem breadth, native network policy constructs and certain Kubernetes-level controls required by this workload.

Region Selection

The deployment is positioned in **us-west-2 (Oregon)** due to:

- Three Availability Zones enabling high-availability topologies.

- Lower average latency to North American users compared to other regions.
- Competitive instance and database pricing compared to EU regions.
- Large instance capacity pools, reducing scheduling latency during autoscaling events.

This platform choice significantly improves scalability, observability, resilience, and security alignment relative to the original VM-based architecture.

C.2 High-Availability Architecture

The production architecture leverages AWS managed services and Kubernetes primitives to achieve resilience across pod, node and Availability Zone failure domains. The design incorporates redundancy, automated healing, secure workload isolation and controlled failover for both stateless and stateful tiers.

Cluster and Network Topology

Multi-AZ EKS Deployment

An EKS cluster is deployed across **three Availability Zones** within us-west-2. Worker nodes run in private subnets, and node groups span all AZs to prevent workload concentration. Pod anti-affinity rules distribute pods across nodes, increasing resilience against node-specific failures.

Application Load Balancer

An ALB operates in public subnets and performs the following functions:

- Distributes traffic across healthy pods.
- Terminates TLS connections.
- Supports weighted routing for blue/green or canary deployments.
- Automatically removes unhealthy pod endpoints, reducing user-visible errors.

RDS MariaDB Multi-AZ

The database tier uses RDS with synchronous Multi-AZ replication. A standby replica in a separate AZ ensures automatic failover within approximately 60 seconds. Backups, patching and storage management are fully automated.

Network Segmentation and Zero Trust Principles

Network architecture follows defence-in-depth:

- **Public subnets:** ALB only

- **Private subnets:** EKS nodes and RDS
- **Network Policies:** enforce workload-to-workload communication boundaries

These constraints reduce attack surface and prevent lateral movement in the event of compromise.

PodSecurityContext and Runtime Hardening

Security controls from Part A are reproduced using Kubernetes-native mechanisms:

Part A Control	Cloud Equivalent
Non-root user	runAsNonRoot: true
Read-only FS	readOnlyRootFilesystem: true
Capability dropping	capabilities: { drop: ["ALL"] }
Secrets via Docker Secrets	Kubernetes Secrets + AWS KMS encryption
AppArmor/seccomp	Pod annotations enforcing restrictive profiles

This maintains consistent security posture across environments.

Failure Scenarios and Expected Behaviour

Failure Scenario	Impact	Automatic Recovery
Pod crash	One replica unavailable	ReplicaSet recreates pod on same or different node
Node failure	All pods on node lost	Pods rescheduled across remaining AZs
AZ outage	Loss of one-third of capacity	ALB routes traffic to healthy pods; nodes in other AZs remain online
Database primary failure	Brief DB unavailability	RDS promotes standby with automatic failover
Traffic surge	Increased load and latency	HPA scales pods; Cluster Autoscaler adds nodes

Failure Scenario	Impact	Automatic Recovery
Rolling deployment	Pod replacement	Zero downtime through progressive rollout

Kubernetes and AWS-managed services together ensure that application downtime is minimised without manual operator intervention.

Disaster Recovery (DR) Strategy

Although Multi-AZ provides high availability, cross-region resilience is addressed through:

- RDS automated backups and point-in-time recovery (PITR).
- Optional read replica in another region for warm DR.
- ECR cross-region replication for container image availability.
- GitHub-hosted source control ensuring invariant application artefacts.

This prepares the system for more severe catastrophic events, aligning with enterprise DR expectations.

C.3 CI/CD Pipeline Design

A secure CI/CD pipeline ensures rapid, reliable and validated deployments while enforcing the security baseline defined in Part A. The proposed workflow uses GitHub Actions, Amazon ECR, and EKS.

```
┌──────────────┐     ┌──────────────┐     ┌──────────────┐     ┌──────────────┐
│ Code         │──▶│   Build      │──▶│   Test       │──▶│   Scan       │
│ Commit       │     │ & Package    │     │ (Unit,       │     │ (Vulnerab.)
│              │     │              │     │ Integ.)      │     │              │
└──────────────┘     └──────────────┘     └──────────────┘     └──────┬───────┘
                                                                      │
┌──────────────┐     ┌──────────────┐     ┌──────────────┐          │
│ Deploy to    │◀────│   Push to    │◀────│   Security   │──────────┘
│ Production   │     │    ECR       │     │    Gate      │
│ (EKS)        │     │              │     │              │
└──────────────┘     └──────────────┘     └──────────────┘
```

C.3.2. Pipeline Implementation 1. Code Commit Stage:

Pipeline Overview

1. Code Commit

A push or pull request triggers the CI pipeline. GitHub branch protection rules enforce review and quality control.

2. Build Stage

Using multi-stage Dockerfiles, hardened images are built with non-root users, minimal base layers and read-only configurations.

```
- name: Build Web Image
  run: docker build -t $ECR_REPO/web:${{ github.sha }} ./webserver
```

3. Test Stage

Unit tests and integration tests validate application logic. Optionally, deployments to an ephemeral EKS namespace enable smoke tests for database connectivity and routing.

4. Supply-Chain and Vulnerability Scanning

Security checks enforce the policies defined in Part A:

- **Trivy** scans OS packages and language dependencies; the pipeline fails on HIGH/CRITICAL findings.
- **Snyk** optionally scans application libraries and IaC configurations.
- **SBOM generation** via Syft provides software transparency.
- **Cosign signing and attestation** ensure only validated artefacts can be deployed.
- **OPA/Conftest policy checks** validate Kubernetes manifests before deployment.

These mechanisms mitigate risks of tampering, dependency compromise and misconfiguration.

5. Push to ECR

Images passing all gates are pushed to ECR using immutable SHA-based tags.

6. Deployment Stage

Two deployment flows:

Direct kubectl apply

GitHub Actions updates image tags in Kubernetes manifests and applies changes:

```
- name: Apply Manifests
  run: kubectl apply -f k8s/
```

GitOps (Recommended Future Enhancement)

Updates are pushed to a manifests repository. **Argo CD** syncs EKS with the desired state, providing drift detection, auditability and controlled rollouts.

7. Post-Deployment Validation

- kubectl rollout status ensures Deployments reach a healthy state.
- Grafana dashboards display latency, error rates and pod health.
- Alertmanager notifies operators if anomalies arise.

Zero-Downtime Deployment Strategies

To avoid service disruption, Kubernetes supports:

- **Rolling updates:** Default Deployment behaviour replaces pods gradually.
- **Blue/Green deployments:** Two environments exist concurrently; traffic shifts only after verification.
- **Canary deployments:** Weighted ALB routing releases new versions to a small user subset first.

Scientific literature confirms these strategies reliably achieve near-zero downtime (Goel et al., 2022).

C.4 Cost Analysis and Justification

C.4.1 Core Cloud Services and Estimated Monthly Cost (GBP)

Summary of Monthly Costs for Target Architecture (us-west-2)

Component	SKU / Pricing Basis	Est. Monthly Cost (£)
EKS Control Plane	£58	£58
Worker Nodes	£72	£72
Node Storage (EBS gp3)	£9–£19	£9–£19
RDS MariaDB Multi-AZ	£52–£65	£52–£65
Application Load Balancer	£19	£19
NAT Gateway (2×)	£52 + data	£52 + data
CloudWatch Logs/Metrics	£8–£24	£8–£24
ECR Storage	£2–£4	£2–£4

Total Monthly Baseline (GBP):

£270–£315 (excluding variable data transfer and log ingestion)

This aligns with published EKS cost analyses showing EC2 nodes, NAT gateways and RDS as the primary contributors (Gupta & Reddy, 2023; CloudZero, 2023).

C.4.2 Explanation of Key Cost Drivers

1. Compute – EC2 Worker Nodes (£72/month)

Worker nodes are the main fixed cost. The selected t3.medium instance type provides efficient general-purpose compute for a small-to-medium workload.

2. Database – RDS MariaDB Multi-AZ (£52–£65/month)

Multi-AZ failover doubles compute cost but is essential for production-grade availability.

3. Networking – NAT & ALB (£71/month + data)

- NAT Gateways (£52 + data): Most expensive network component due to hourly and per-GB fees.
- Application Load Balancer (£19): Low fixed cost but scales with traffic.

4. Logging & Monitoring – CloudWatch (£8–£24/month)

Costs vary with log volume and retention configuration.

C.4.3 Cost-Optimisation Strategies (Financial Impact Reflected in GBP)

Reserved Instances & Savings Plans

- Reduce EC2/RDS compute costs by 40–60% (AWS, 2024i).
- This brings worker node cost from £72 → ~£30–£40 and RDS from £52–£65 → ~£20–£30.

Autoscaling & Right-Sizing

- Kubernetes autoscaling ensures nodes operate only when required, preventing idle spend.
- Minimising the cluster to 2 nodes off-peak reduces compute cost by ~£20/month.

Spot Instances (Up to 70% Cost Reduction)

- Ideal for staging, batch tasks and CI pipelines.
- Example A t3.medium Spot instance typically costs £18/month instead of £24/month.

Serverless Alternatives (Lambda/Fargate)

- Eliminates always-on compute costs for background or scheduled jobs.
- Only pay for execution time; cost reductions vary based on workload profile.

Aurora Serverless v2 (Future Option)

- Database capacity scales automatically, costs become usage based.
- Beneficial when workload patterns are unpredictable.

NAT Cost Reduction via VPC Endpoints

- Moving S3/ECR/CloudWatch traffic to PrivateLink removes NAT data fees entirely.
- Typical NAT savings: £10–£40 per month, depending on workload.

Log Lifecycle Management

- Reducing retention and filtering noisy logs lowers CloudWatch bills by £5–£15/month.

C.4.4 Cost Justification

Despite the monthly cost of £270–£315, the architecture provides:

- Managed control plane and database operations, reducing risk and admin overhead (Concurrency Labs, 2023).
- High availability through multi-AZ, impossible to achieve on a single-VM setup.
- Integrated observability and security, aligning with the hardened design from Part A.
- Elasticity, ensuring the system only pays for required capacity.

Research on managed Kubernetes platforms confirms that once operational overhead, downtime avoidance and resilience are considered, EKS-based architectures often achieve lower total cost of ownership compared with VM-only deployments (Sharma et al., 2024; Gupta & Reddy, 2023).

References

Alzahrani, N., Alqahtani, M. and Alshamrani, A. (2022) 'Detection, analysis and countermeasures for container-based misconfiguration using Docker and Kubernetes', *IEEE Access*, 10, pp. 45873–45889. Available at: https://ieeexplore.ieee.org/document/9754093 (Accessed: 10 December 2025).

Centre for Internet Security (CIS) (2023) *CIS Docker Benchmark v1.6.0*. Available at: https://www.cisecurity.org/benchmark/docker (Accessed: 03 December 2025).

CloudZero (2023) *Understanding Amazon EKS costs*. Available at: https://www.cloudzero.com/blog/eks-cost/ (Accessed: 06 December 2025).

Concurrency Labs (2023) *Operational benefits of managed Kubernetes*. Available at: https://www.concurrencylabs.com/blog/managed-kubernetes/ (Accessed: 03 December 2025).

Docker Inc. (2024) *Docker Compose documentation*. Available at: https://docs.docker.com/compose/ (Accessed: 13 December 2025).

Docker Inc. (2024) *Docker security best practices*. Available at: https://docs.docker.com/engine/security/ (Accessed: 13 December 2025).

Goel, A., Chhetri, M.B. and Kowalczyk, R. (2022) 'Comparison of zero-downtime based deployment techniques in public cloud infrastructure', *Proceedings of the IEEE Conference on Cloud Computing*. Available at: https://ieeexplore.ieee.org/document/9814873 (Accessed: 12 December 2025).

Gupta, R. and Reddy, S. (2023) 'Cloud-native DevSecOps cost and performance evaluation', *Journal of Cloud Computing*, 12(1), pp. 1–18. Available at: https://journalofcloudcomputing.springeropen.com/articles/10.1186/s13677-023-00382-1 (Accessed: 11 December 2025).

Kubernetes (2024) *Security contexts*. Available at: https://kubernetes.io/docs/tasks/configure-pod-container/security-context/ (Accessed: 13 December 2025).

National Institute of Standards and Technology (NIST) (2020) *SP 800-190: Application Container Security Guide*. Available at: https://csrc.nist.gov/publications/detail/sp/800-190/final (Accessed: 07 December 2025).

OWASP (2021) *OWASP Top 10 Container Security Risks*. Available at: https://owasp.org/www-project-top-10-container-security-risks/ (Accessed: 01 December 2025).

Prometheus (2024) *Alertmanager documentation*. Available at: https://prometheus.io/docs/alerting/latest/alertmanager/ (Accessed: 14 December 2025).

Sharma, P. and Sood, K. (2021) 'Securing container-based virtualization platforms', *International Journal of Information Security*, 20(4), pp. 421–438. Available at: https://link.springer.com/article/10.1007/s10207-021-00533-8 (Accessed: 02 December 2025).

Sharma, P., Gupta, R. and Reddy, S. (2024) 'DevSecOps practices in cloud-native environments', *ACM Computing Surveys*, 56(2), pp. 1–36. Available at: https://dl.acm.org/doi/10.1145/3611234 (Accessed: 02 December 2025).

Singh, A., Dutta, K. and Chatterjee, P. (2020) 'Securing vulnerabilities in Docker images', *International Journal of Cloud Applications and Computing*, 10(3), pp. 45–62. Available at: https://www.igi-global.com/article/securing-vulnerabilities-in-docker-images/252492 (Accessed: 08 December 2025).

Tu, Z., Li, H. and Zhang, Y. (2021) 'Lightweight container image hardening using multi-stage builds', *IEEE Access*, 9, pp. 135421–135433. Available at: https://ieeexplore.ieee.org/document/9537844 (Accessed: 03 December 2025).

Wazuh (2024) *Security monitoring and SIEM*. Available at: https://documentation.wazuh.com/ (Accessed: 13 December 2025).

Amazon Web Services, Inc. (2024a) *Amazon Elastic Kubernetes Service (EKS) pricing*. Available at: https://aws.amazon.com/eks/pricing/ (Accessed: 14 December 2025).

Amazon Web Services, Inc. (2024b) *Amazon EC2 on-demand pricing*. Available at: https://aws.amazon.com/ec2/pricing/on-demand/ (Accessed: 12 December 2025).

Amazon Web Services, Inc. (2024c) *Amazon EBS gp3 pricing*. Available at: https://aws.amazon.com/ebs/pricing/ (Accessed: 11 December 2025).

Amazon Web Services, Inc. (2024d) *Amazon RDS for MariaDB pricing*. Available at: https://aws.amazon.com/rds/mariadb/pricing/ (Accessed: 07 December 2025).

Amazon Web Services, Inc. (2024e) *Elastic Load Balancing pricing*. Available at: https://aws.amazon.com/elasticloadbalancing/pricing/ (Accessed: 08 December 2025).

Amazon Web Services, Inc. (2024g) *Amazon CloudWatch pricing*. Available at: https://aws.amazon.com/cloudwatch/pricing/ (Accessed: 14 December 2025).

Amazon Web Services, Inc. (2024h) *Amazon Elastic Container Registry (ECR) pricing*. Available at: https://aws.amazon.com/ecr/pricing/ (Accessed: 14 December 2025).

Amazon Web Services, Inc. (2024i) *Savings Plans and Reserved Instances*. Available at: https://aws.amazon.com/savingsplans/ (Accessed: 10 December 2025).